Practice answer key

Introduction and Chapters 1-5

Worked solutions to all 112 questions in the accompanying printable guide. Calculations, register traces, floating-point fields, pipeline schedules, cache states, and example MIPS programs explain how to reach each result.

Try each problem before reading its solution. For code questions, equivalent correct implementations are possible. The examples here emphasize explicit steps and core instructions.

Print specification	Setting
Paper	A4; print at actual size / 100%
Main text, tables, code	12 pt Candara
Lesson / section headings	19.2 / 21.6 pt
Subheadings	14.4 pt
Source notes / running text	10.2 / 9.2 pt
Color	Pure black and white

Prepared from the supplied slide decks. 14 October 2026.

Contents

Solutions | Introduction and performance 7

Q01. What is architecture? . . . 7
Q02. Find the computer class and component . . . 7
Q03. Connect the representations . . . 7
Q04. Interpret a technology claim . . . 7
Q05. Distinguish the kinds of time . . . 7
Q06. Relative performance and percentages . . . 8
Q07. Clock units . . . 8
Q08. A redesign changes the cycle count . . . 8
Q09. The full CPU-time equation . . . 8
Q10. Weighted CPI . . . 8
Q11. A partial improvement . . . 8
Q12. Dynamic power ratios . . . 9
Q13. Parallelism with a serial portion . . . 9
Q14. A benchmark summary . . . 9

Solutions | Values, memory, and control flow 10

Q15. Binary and hex . . . 10
Q16. The same byte, two interpretations . . . 10
Q17. A register trace . . . 10
Q18. Translate an expression . . . 10
Q19. Addresses and alignment . . . 11
Q20. Load widths and extension . . . 11
Q21. Partial stores, executed in sequence . . . 11
Q22. Big-endian and little-endian layouts . . . 12
Q23. A constant array index . . . 12
Q24. A variable array index . . . 12
Q25. A nested array index . . . 12
Q26. Valid shifts and a malformed instruction . . . 13
Q27. Bitwise operations . . . 13
Q28. Build a full-word constant . . . 13
Q29. Follow a loop and count its branches . . . 13
Q30. Translate a signed if / else . . . 14

Q31. Signed and unsigned comparison . . . 14
Q32. A sign-extended unsigned comparison . . . 14
Q33. A bounded search loop . . . 14
Q34. A for loop that accumulates a sum . . . 15
Q35. Expand conveniences . . . 15

Solutions | Procedures and strings **16**

Q36. Calls in two execution models . . . 16
Q37. Who preserves the value? . . . 16
Q38. A leaf procedure using a saved register . . . 16
Q39. A stack-frame address calculation . . . 16
Q40. Repair a non-leaf return . . . 17
Q41. Trace recursive factorial . . . 17
Q42. Copy a terminator . . . 17

Solutions | Machine instruction encodings **18**

Q43. Choose formats . . . 18
Q44. Encode R-format instructions . . . 18
Q45. Encode I-format instructions . . . 19
Q46. Encode backward and forward branches . . . 19
Q47. A jump at an address-region boundary . . . 19
Q48. Encode and trace a complete sequence . . . 20

Solutions | Chapter 3 arithmetic **21**

Q49. Range and interpretation . . . 21
Q50. Carry and signed overflow . . . 21
Q51. Subtraction using two's complement . . . 21
Q52. Trapping versus non-trapping arithmetic . . . 21
Q53. Partial-product multiplication . . . 22
Q54. A shift-and-add trace . . . 22
Q55. Unsigned HI and LO . . . 22
Q56. Signed multiplication fit . . . 23
Q57. Signed quotient and remainder . . . 23
Q58. Binary long division . . . 23
Q59. Special-result registers and division safety . . . 23
Q60. Normalize a binary fraction . . . 24

Q61. Decode IEEE-754 . . . 24
Q62. Encode IEEE-754 . . . 24
Q63. Reserved floating-point patterns . . . 24
Q64. Ties-to-even rounding . . . 24
Q65. Floating-point addition steps . . . 25
Q66. Floating-point multiplication steps . . . 25
Q67. FP comparison and control flow . . . 25
Q68. FP instruction fields . . . 25

Solutions | Chapter 4 microarchitecture **26**

Q69. Follow a load through five stages . . . 26
Q70. MUX selection and write enable . . . 26
Q71. Extract fields versus register contents . . . 26
Q72. Load path and control . . . 26
Q73. Store path and control . . . 27
Q74. Branch signal versus taken signal . . . 27
Q75. Two levels of ALU decoding . . . 27
Q76. Single-cycle critical path . . . 27
Q77. Finite pipeline speedup . . . 27
Q78. Which instruction controls cycle 5? . . . 28
Q79. Preserve the destination number . . . 28
Q80. RAW without forwarding . . . 28
Q81. Fill a load-use gap safely . . . 28
Q82. Forward from two different boundaries . . . 29
Q83. Choose the latest forwarded value . . . 29
Q84. An immediate load-use pair . . . 29
Q85. Repair incorrect stall control . . . 29
Q86. A field match that is not a source dependency . . . 29
Q87. Branch penalties and the endpoint . . . 30
Q88. A one-bit predictor . . . 30

Solutions | Chapter 5 memory hierarchy **31**

Q89. Identify locality . . . 31
Q90. Set count and address widths . . . 31
Q91. Map a specific byte address . . . 31

Q92. Direct-mapped state trace . . . 31
Q93. Valid is part of the hit test . . . 31
Q94. Include metadata storage . . . 32
Q95. Placement choices . . . 32
Q96. LRU updates on hits . . . 32
Q97. Dirty write-back behavior . . . 32
Q98. Allocation is separate from write propagation . . . 33
Q99. Block-transfer bandwidth . . . 33
Q100. Miss contributions to CPI . . . 33
Q101. AMAT and miss-time conventions . . . 33
Q102. Local and global L2 miss rates . . . 33
Q103. More GHz does not halve a fixed memory wait . . . 34
Q104. Matrix layout and tile footprint . . . 34
Q105. Virtual-to-physical translation . . . 34
Q106. Change the page size . . . 34
Q107. A flat page-table size . . . 34
Q108. Three different misses . . . 35
Q109. A small TLB LRU trace . . . 35
Q110. A dirty page victim . . . 35
Q111. Classify the three Cs . . . 35
Q112. Combine translation and a data-cache trace . . . 36

How to use these solutions

The question numbers match the main guide. Each solution gives the reason for the result, not just a final value. For programming questions, trace both control paths or several loop iterations to check an alternative solution.

Conventions: 32-bit MIPS; big-endian memory; no delay slots unless explicitly discussed; a word is 4 bytes. Signed arithmetic examples are within range. Classic delayed MIPS requires delay-slot handling and uses PC + 8 for jal. All encodings use actual instruction fields, including PC + 4 in branch and jump target reconstruction.

The supplied reference card and instruction-set manual were used to verify instruction semantics and encodings. Source slide locations and clarification notes are in the main guide.

Solutions | Introduction and performance

Q01. What is architecture?

Related lessons: I.1 and 2.1.

The ISA defines programmer-visible behavior, such as instructions, registers, and addressing rules. An implementation uses particular circuits, timing, caches, and other resources to deliver that behavior. Two processors can execute the same ISA with different CPI or clock frequencies, so their CPU times can differ even for the same instruction sequence.

Q02. Find the computer class and component

Related lesson: 1.2.

The laptop is a personal computer. The washing-machine controller is embedded. The network file server is a server. A large scientific simulation system can be a supercomputer; the workload description alone does not force every scientific system to be one, so a reasonable classification should explain its scale.

The CPU's arithmetic datapath performs calculations. Main memory holds active program data. Input hardware and its controller connect keyboard input to the system. Registers hold immediately used CPU operands; storage holds longer-term files.

Q03. Connect the representations

Related lesson: 1.3.

The conceptual translation order is high-level code, Assembly, then machine code. A compiler translates the high-level program; an assembler encodes Assembly and resolves its labels. Some compilers emit machine code without a separate Assembly file. The operating system manages resources and services when the compiled program runs. A label names an address; it is not an executed machine instruction.

Q04. Interpret a technology claim

Related lesson: 1.1.

Three doublings give 500 x 2 x 2 x 2 = **4,000**. Transistor count measures integration, while program time depends on the algorithm, instruction count, CPI, frequency, and other costs. More transistors may be used for cache or extra cores, which do not necessarily accelerate every part of a particular program.

Q05. Distinguish the kinds of time

Related lesson: 1.4.

Original elapsed time is 4 + 5 + 2 = **11 seconds**. CPU time is 4 + 2 = **6 seconds**. After halving computation, elapsed time is 2 + 5 + 1 = **8 seconds**. The I/O wait remains 5 seconds. Tasks completed per hour measures **throughput**, rather than the response time of one task.

Q06. Relative performance and percentages

Related lesson: 1.5.

Performance A / performance B = time B / time A = 18/12 = **1.5**. The performance increase from B to A is (1.5 - 1) x 100% = **50%**. The time reduction is (18 - 12)/18 x 100% = **33.33%**. The percentages differ because time and performance are reciprocal quantities.

Q07. Clock units

Related lesson: 1.6.

T = 1/(2.5 x 10^9) seconds = **0.4 ns**. Time for 4,000 cycles is 4,000 x 0.4 ns = 1,600 ns = **1.6 microseconds**. The units cancel as cycles x seconds/cycle.

Q08. A redesign changes the cycle count

Related lesson: 1.6.

A's cycles are 8 x 3 x 10^9 = **24 billion**. B's cycles are 1.25 x 24 billion = **30 billion**. Its required frequency is 30 billion cycles / 5 seconds = **6 GHz**. Changing only the time ratio would miss B's extra cycle requirement.

Q09. The full CPU-time equation

Related lesson: 1.7.

Cycles = 1.2 x 10^6 x 1.75 = **2.1 million cycles**. Time = (2.1 x 10^6)/(2.1 x 10^9) seconds = 0.001 s = **1 ms**. A GHz number prices each cycle in seconds, but IC and CPI determine how many cycles must execute.

Q10. Weighted CPI

Related lesson: 1.8.

Class	Count	CPI	Cycles
A	4,000	1	4,000
B	3,500	2	7,000
C	2,500	4	10,000
Total	10,000		21,000

Average CPI = 21,000/10,000 = **2.1**. Time = 21,000/(3 x 10^9) seconds = **7 microseconds**. Equivalently, 0.40 x 1 + 0.35 x 2 + 0.25 x 4 = 2.1.

Q11. A partial improvement

Related lesson: 1.9.

Class C now costs 2,500 x 2 = 5,000 cycles. Total cycles = 4,000 + 7,000 + 5,000 = **16,000**. Average CPI = **1.6**. CPU time = 16,000/(3 x 10^9) = **5.3333 microseconds**. Speedup = 21,000/16,000 = **1.3125**.

Class C originally used 10,000/21,000 = **10/21, about 47.62%** of the cycles. It was 25% of the instructions, but not 25% of the execution time. With unchanged frequency, cycle fractions equal CPU-time fractions. Amdahl's expression with f = 10/21 and s = 2 gives 1 / (11/21 + 5/21) = 21/16, matching the direct calculation.

Q12. Dynamic power ratios

Related lesson: 1.10.

The ratio is 0.90 x 0.80^2 x 1.10 = **0.6336**. Reduction = (1 - 0.6336) x 100% = **36.64%**. New dynamic power = 80 x 0.6336 = **50.688 W**. This calculation assumes the stated dynamic-power model and unchanged switching activity; it does not estimate all components of total chip power.

Q13. Parallelism with a serial portion

Related lesson: 1.11.

Normalized new time = 0.25 + 0.75/4 = **0.4375**. Speedup = 1/0.4375 = **16/7, about 2.2857**. The serial quarter still takes its original time. Perfect division of the remaining work cannot eliminate that quarter.

Q14. A benchmark summary

Related lesson: 1.12.

The ratios are 100/50 = **2**, 120/30 = **4**, and 80/10 = **8**. Their product is 64; its cube root is **4**. Measured seconds are workload-dependent raw times. The requested summary combines normalized performance ratios with a geometric mean, not an arithmetic mean of raw times.

Solutions | Values, memory, and control flow

Q15. Binary and hex

Related lesson: 2.2.

Hex 6 is binary 0110 and hex D is 1101, so 0x6D = **01101101**. Its set-bit weights are 64 + 32 + 8 + 4 + 1 = **109**. A 32-bit word contains **4 bytes**, because each byte has 8 bits.

Q16. The same byte, two interpretations

Related lessons: 2.2 and 2.6.

0xD6 = 13 x 16 + 6 = **214** unsigned. The top bit is 1, so signed 8-bit value = 214 - 256 = **-42**. Sign extension gives **0xFFFFFFD6**; zero extension gives **0x000000D6**. A 32-bit 0xFFFFFFFF is **-1** signed or **4,294,967,295** unsigned.

Q17. A register trace

Related lesson: 2.4.

Point	$t0	$t1	$t2
Initially	Unknown	8	3
After add	11	8	3
After sub	11	8	3
After addi	11	8	6

The subtraction reads t0 = 11 and t2 = 3, producing 8 in t1. The final addi uses that current t1 and immediate -2, producing 6.

Q18. Translate an expression

Related lesson: 2.4.

```
sub  $t0, $s1, $s2    # g - h
addi $t1, $s3, 6      # i + 6
add  $s0, $t0, $t1    # f
```

For g = 10, h = 4, i = 3, the temporary values become 6 and 9, so **f = 15**. Registers s1, s2, s3 retain their input values. Other equivalent code is valid if it preserves the required inputs and implements the same expression.

Q19. Addresses and alignment

Related lesson: 2.5.

Instruction	Effective address	Alignment
lw with offset 12	2048 + 12 = 2060	Valid: divisible by 4
lhu with offset 3	2048 + 3 = 2051	Invalid: not divisible by 2
sb with offset 7	2048 + 7 = 2055	Valid: byte access

The halfword load raises an alignment error in the model. Loaded contents cannot be determined for the valid loads either, because no memory contents were supplied.

Q20. Load widths and extension

Related lessons: 2.6-2.7.

Destination	Read	Result
$t0, lw	80 F1 7F 00	0x80F17F00
$t1, lh	80 F1, signed halfword	0xFFFF80F1
$t2, lb	80, signed byte	0xFFFFFF80
$t3, lbu	80, unsigned byte	0x00000080
$t4, lhu at offset 2	7F 00, unsigned halfword	0x00007F00

The load widths decide how many bytes are combined. The signed or unsigned load variant decides how the remaining register bits are filled.

Q21. Partial stores, executed in sequence

Related lessons: 2.6-2.7.

Address	Final byte, hex	Cause
6000	12	sw, most significant byte
6001	34	sw
6002	AB	sw
6003	CD	sw, least significant byte
6004	CD	sb, low 8 bits
6005	00	Never written
6006	AB	sh, upper byte of low halfword
6007	CD	sh, lower byte of low halfword

Final bytes are **12 34 AB CD CD 00 AB CD**. $t0 stays **0x1234ABCD**. Stores copy from the register; they do not modify it.

Q22. Big-endian and little-endian layouts

Related lesson: 2.7.

Layout	Bytes at 100, 101, 102, 103	lb at 101
Big-endian	90 AB CD EF	0xFFFFFFAB
Little-endian	EF CD AB 90	0xFFFFFFCD

Both bytes at address 101 have their top bit set, so lb sign-extends. The difference comes from which byte occupies that address, not from a different extension rule.

Q23. A constant array index

Related lesson: 2.8.

```
lw  $t0, 8($s3)      # 2 x 4
add $t0, $s2, $t0
sw  $t0, 28($s3)     # 7 x 4
```

The offsets are **8** and **28** bytes. The sum is 9 + 13 = **22**, stored in A[7]. A[2], h, and the array base stay unchanged.

Q24. A variable array index

Related lesson: 2.9.

```
sll  $t0, $s1, 2
addu $t0, $s6, $t0
lw   $t1, 0($t0)
addi $s0, $t1, 1
```

The displacement is 3 x 4 = **12**, the address is 1000 + 12 = **1012**, the loaded value is **20**, and f becomes **21**. The array is read, not changed. Address addition uses addu to avoid treating a valid address bit pattern as signed-overflow arithmetic.

Q25. A nested array index

Related lesson: 2.9.

```
lw   $t0, 8($s7)     # B[2]
sll  $t0, $t0, 2     # its value is an element index into A
addu $t0, $s6, $t0
lw   $t1, 0($t0)
sub  $s0, $s1, $t1
```

The first load yields t0 = **4**, an index. The shift gives **16**, a byte displacement. Adding s6 gives the **address of A[4]**. The second load yields t1 = **7**, a value. Finally f = 50 - 7 = **43**.

Q26. Valid shifts and a malformed instruction

Related lesson: 2.10.

sll by 4 gives **0x00000110**. The high hex digit 8 is discarded from the 32-bit result; a zero enters on the right. srl by 4 gives **0x08000001**; a zero enters on the left, and the low hex digit 1 is discarded.

An ordinary immediate sll with shift amount **44 is invalid**, because shamt has 5 bits and represents only 0-31. The question does not authorize guessing that 44 should mean 12 or 4.

Q27. Bitwise operations

Related lesson: 2.11.

Operation	Result
to AND t1	0x00340078
to OR t1	0x12FF56FF
to NOR zero	0xEDCBA987
to AND zero-extended 00F0	0x00000070

For the low byte 0x78 AND 0xF0: 01111000 AND 11110000 = 01110000, or **0x70**. Complementing each nibble for NOR with zero changes 1 to E, 2 to D, and so on. The andi result's upper 16 bits are zero because the immediate is zero-extended.

Q28. Build a full-word constant

Related lessons: 2.6 and 2.11.

```
lui $t0, 0x89AB
ori $t0, $t0, 0xCDEF
```

After lui, t0 is **0x89AB0000**. After ori, it is **0x89ABCDEF**. Neither instruction reads memory. The lui mnemonic includes "load" but its operand is an immediate, not a memory address.

Q29. Follow a loop and count its branches

Related lesson: 2.12.

Trip	t0 after increment	bne decision
1	1	Taken
2	2	Taken
3	3	Not taken

Final t0 = **3**; final t2 = 3 + 3 = **6**. The branch executes **3** times and is taken **2** times. Dynamic instruction count = 2 initializations + 3 x (increment + branch) + 1 final add = **9**. The label adds no instruction to the count.

Q30. Translate a signed if / else

Related lessons: 2.13-2.14.

```
slti $t0, $s0, 5      # a < 5
bne  $t0, $zero, Else
add  $s0, $s0, $s1    # a >= 5
j    Done
Else: sub $s0, $s0, $s1
Done:
```

For (4, 3), slti produces 1, the branch is taken, and a becomes **1**. For (5, 3), slti produces 0, the branch is not taken, and a becomes **8**. The jump after addition prevents execution of the subtraction path.

Q31. Signed and unsigned comparison

Related lesson: 2.14.

Signed slt interprets FFFFFFFF as -1. Since -1 < 1, **t2 = 1**. Unsigned sltu interprets it as 4,294,967,295. That is not less than 1, so **t3 = 0**. Both instructions read the same bits; their interpretation differs.

Q32. A sign-extended unsigned comparison

Related lesson: 2.14.

Immediate -1 has 16-bit pattern FFFF and sign-extends to **0xFFFFFFFF**. sltiu compares unsigned 65,536 against 4,294,967,295, so **t1 = 1**. An incorrect zero-extension assumption would compare 65,536 against 65,535 and incorrectly produce 0.

Q33. A bounded search loop

Related lesson: 2.15.

```
addi $s1, $zero, 0
Loop:
slt  $t0, $s1, $s3
beq  $t0, $zero, Done  # stop before an out-of-bounds load
sll  $t1, $s1, 2
addu $t1, $s0, $t1
lw   $t2, 0($t1)
bne  $t2, $s2, Done
addi $s1, $s1, 1
j    Loop
Done:
```

For [4, 4, 9], the first two elements match; the third does not. The loop exits with **i = 2**. If all three elements equal 4, it increments i to 3; the next bounds test fails and exits with **i = 3 without loading A[3]**. Testing the bound first preserves the high-level && short-circuit behavior.

Q34. A for loop that accumulates a sum

Related lesson: 2.15.

```
addi $s0, $zero, 0
addi $s1, $zero, 0
Loop:
slti $t0, $s1, 4
beq  $t0, $zero, Done
add  $s0, $s0, $s1
addi $s1, $s1, 1
j    Loop
Done:
```

i used in the body	Sum before body	Sum after body
0	0	0
1	0	1
2	1	3
3	3	6

After the fourth update, i = 4. The test then fails. Final **sum = 6** and **i = 4**. Using a test of i <= 4 would execute an extra body and give a different sum.

Q35. Expand conveniences

Related lesson: 2.16.

```
# move $t0, $t1
addu $t0, $t1, $zero

# blt $t0, $t1, Less
slt  $at, $t0, $t1
bne  $at, $zero, Less
```

The branch expansion uses **$at**, the assembler temporary. One pseudo line can expand to two machine words, so later instruction addresses and branch displacements must be calculated after expansion. Alternative equivalent move implementations are acceptable.

Solutions | Procedures and strings

Q36. Calls in two execution models

Related lesson: 2.17.

With jal at 4000, the no-delay-slot model saves **4004**. Classic delayed MIPS saves **4008** and executes the delay-slot instruction at **4004** before entering the callee. jr $ra changes control to the address stored in ra; a classic delayed jr also has its own delay slot.

Q37. Who preserves the value?

Related lesson: 2.18.

The callee must restore **s0 = 77** if it changed s0. It is allowed to clobber t0. The caller must arrange to save its old **t0 = 9** if it needs that value after the call, for example in caller-owned stack space. Preservation is performed by program instructions following a convention, not automatically by the CPU based on the register name.

Q38. A leaf procedure using a saved register

Related lesson: 2.19.

```
SumPlusOne:
addiu $sp, $sp, -8
sw    $s0, 0($sp)
add   $s0, $a0, $a1
addi  $s0, $s0, 1
addu  $v0, $s0, $zero
lw    $s0, 0($sp)
addiu $sp, $sp, 8
jr    $ra
```

The frame starts at sp = **992** and saves 55 there. The body computes s0 = 30, then 31, and copies **31 to v0**. At return, **s0 = 55** and **sp = 1000**. The procedure is a leaf and makes no nested call, so it does not overwrite ra and need not save it.

Q39. A stack-frame address calculation

Related lesson: 2.18.

Reserving 16 decimal bytes subtracts hex 0x10: new sp = 0x1000 - 0x10 = **0x0FF0**. The store offset 12 is hex 0xC, so the saved word is at 0x0FF0 + 0xC = **0x0FFC**. After releasing the whole frame, sp must be **0x1000**.

Q40. Repair a non-leaf return

Related lesson: 2.20.

jal Inner replaces Outer's incoming ra with the address of its own following jr. Inner returns there. That jr then jumps back to itself in the no-delay-slot fragment instead of returning to Outer's caller. Save the incoming continuation before the nested call:

```
Outer:
addiu $sp, $sp, -8
sw    $ra, 4($sp)
jal   Inner
lw    $ra, 4($sp)
addiu $sp, $sp, 8
jr    $ra
```

The unused frame word is padding. The lw retrieves Outer's original return address, and sp is balanced before returning. A classic delayed version would also need safe delay-slot instructions.

Q41. Trace recursive factorial

Related lesson: 2.20.

Calls descend through **fact(3), fact(2), fact(1), fact(0)**. The base returns 1; unwinding returns 1 for n = 1, 2 for n = 2, and **6 for n = 3**. The maximum is **4 simultaneous frames**, each 8 bytes, for **32 bytes**. Initial decimal sp 1000 reaches **968**. After all four frames are released, the caller receives **v0 = 6 and sp = 1000**.

Each non-base frame needs its own original n and original return address. Restoring only the last call's n or ra would lose the data required by earlier pending invocations.

Q42. Copy a terminator

Related lesson: 2.21.

Final destination bytes are **41 42 00 FF**. Exactly **3 bytes** are copied using indexes **0, 1, 2**. The fourth byte is untouched. Copying zero before testing ensures the destination is terminated. A version that exits before the zero store must explicitly write a terminator before it returns.

Solutions | Machine instruction encodings

Q43. Choose formats

Related lesson: 2.22.

Instruction	Format
add	R
sll	R
lw	I
bne	I
j	J
jr	R
lui	I

R widths are **6, 5, 5, 5, 5, 6**. I widths are **6, 5, 5, 16**. J widths are **6, 26**. sll uses its R-format shamt field for the shift amount, rather than a 16-bit I-format immediate field. The operation's specified layout decides its format.

Q44. Encode R-format instructions

Related lesson: 2.23.

For sub $t2, $s0, $s1: opcode = 0, rs = 16, rt = 17, rd = 10, shamt = 0, funct = 34. Binary fields are:

```
000000 10000 10001 01010 00000 100010
Hex word: 0x02115022
```

For sll $t0, $t1, 3: opcode = 0, rs = 0, rt = 9, rd = 8, shamt = 3, funct = 0.

```
000000 00000 01001 01000 00011 000000
Hex word: 0x000940C0
```

The shift's input is in rt. Placing it in rs would encode a different field pattern.

Q45. Encode I-format instructions

Related lesson: 2.24.

For addi $s0, $s0, -8: opcode = 8, rs = 16, rt = 16. The 16-bit immediate is 65,536 - 8 = 65,528 = **0xFFF8**.

001000 10000 10000 1111111111111000
Hex word: 0x2210FFF8

For sw $t2, 12($s1): opcode = 43, rs = 17, rt = 10, immediate = 12.

101011 10001 01010 0000000000001100
Hex word: 0xAE2A000C

rs is the base register; rt is the stored source. The current numeric contents of s1 and t2 are not encoded in the instruction word.

Q46. Encode backward and forward branches

Related lesson: 2.25.

For bne at 0x100C: P + 4 = **0x1010**. Byte difference = 0x1000 - 0x1010 = **-16**. Displacement = -16/4 = **-4**, represented by **0xFFFC**. Fields are op 5, rs 8, rt 9, immediate FFFC. The word is **0x1509FFFC**.

For beq at 0x2000: P + 4 = **0x2004**. Byte difference = 0x2010 - 0x2004 = **12**. Displacement = 12/4 = **3**. Fields are op 4, rs 16, rt 17, immediate 0003. The word is **0x12110003**.

Reconstruction checks: 0x1010 + 4 x (-4) = 0x1000; 0x2004 + 4 x 3 = 0x2010.

Q47. A jump at an address-region boundary

Related lesson: 2.26.

P + 4 = **0x90000000**, so reconstruction uses upper nibble **9**, even though the jump instruction's own address begins with 8. Target index = (0x90000040 >> 2) & 0x03FFFFFF = **0x00000010**. Opcode 2 occupies the top 6 bits, producing **0x08000010**.

Reconstruct: 0x90000000 | (0x10 << 2) = **0x90000040**. The same instruction address cannot directly target **0x80000040** through J-format, because P + 4 selects region 9. A full address in a register with an appropriate indirect jump would be needed instead.

Q48. Encode and trace a complete sequence

Related lessons: 2.27-2.28.

Address	Assembly	Fields	Machine word
0x00400000	addi $t0, $zero, 3	op 8; rs 0; rt 8; imm 3	0x20080003
0x00400004	addi $t0, $t0, -1	op 8; rs 8; rt 8; imm FFFF	0x2108FFFF
0x00400008	bne $t0, $zero, Loop	op 5; rs 8; rt 0; imm FFFE	0x1500FFFE
0x0040000C	j Done	op 2; index 0x00100004	0x08100004
0x00400010	add $s0, $t0, $zero	rs 8; rt 0; rd 16; fn 32	0x01008020

For bne, the reference address is 0x0040000C. Loop is 8 bytes earlier, so the displacement is **-2**. For j, Done/4 gives target index **0x00100004**, with the zero upper address region supplied by P + 4.

The loop decrements $t0 from 3 to **2, then 1, then 0**. The branch executes **3 times**, taken for 2 and 1, and not taken for 0. The jump reaches Done, which sets **$s0 = 0**. Other correct encodings must decode to the same register roles, immediates, and targets.

Solutions | Chapter 3 arithmetic

Q49. Range and interpretation

Related lessons: 3.1-3.3.

1110 has unsigned value 8 + 4 + 2 = **14**. Its top bit is 1, so signed four-bit value is 14 - 16 = **-2**. Unsigned range is 0..15; signed range is -8..7. For -5, start with +5 = 0101, invert to 1010, then add 1: **1011**. Positive 8 would require 1000, which is the pattern for signed -8, so +8 is outside the signed range.

Q50. Carry and signed overflow

Related lesson: 3.2.

0110 + 0101 = **1011**, exact unsigned sum 6 + 5 = 11. There is no carry beyond four bits. Signed 1011 means 11 - 16 = **-5**, so signed overflow occurred: two positive inputs produced a negative stored result, and the exact +11 exceeds +7. By contrast, 1111 + 0001 gives 1 0000. The carry is discarded, but signed -1 + 1 = 0 is in range. Carry out tests unsigned range, not signed range.

Q51. Subtraction using two's complement

Related lesson: 3.3.

-6 is 1010; +5 is 0101. Negate +5 by inversion and addition: 1010 + 1 = 1011, representing -5. Then 1010 + 1011 = 1 0101, so the stored four-bit result is **0101, +5**. The exact answer is **-11**, below the signed minimum -8. A negative operand minus a positive operand produced a nonnegative stored result, confirming signed overflow.

Q52. Trapping versus non-trapping arithmetic

Related lesson: 3.4.

addi tries to compute +2,147,483,647 + 1. The result cannot fit signed 32 bits, so it raises a signed overflow exception rather than committing the ordinary result to t1. addiu produces low bits **0x80000000** without that signed overflow exception. The pattern is signed -2,147,483,648 if interpreted as signed. addiu **sign-extends** its 16-bit immediate. Standard unsigned C arithmetic wraps; ordinary signed C overflow is undefined, so hardware wrap behavior is not a language guarantee.

Q53. Partial-product multiplication

Related lesson: 3.5.

The multiplicand is 13 and multiplier is 11 = 8 + 2 + 1. Multiplier bits 0, 1, and 3 contribute rows; bit 2 contributes zero.

```
  00001101      13 << 0
  00011010      13 << 1
  00000000       multiplier bit 2 is zero
+ 01101000      13 << 3
----------
  10001111      143 = 0x8F
```

143 exceeds unsigned four-bit maximum 15. Eight bits are enough because two four-bit inputs can require an eight-bit product.

Q54. A shift-and-add trace

Related lesson: 3.6.

Rows below show state after both shifts. Values are decimal for easy checking; use an eight-bit P/M and four-bit Q.

Iteration	Q low bit before step	Addition	P afterward	M afterward	Q afterward
Initial	-	-	0	3	5
1	1	0 + 3	3	6	2
2	0	None	3	12	1
3	1	3 + 12	15	24	0
4	0	None	15	48	0

Final P is **15**, binary 00001111, agreeing with 3 x 5. Testing the bit after shifting would change which multiplicand weight is included and is not this algorithm.

Q55. Unsigned HI and LO

Related lesson: 3.7.

(2^32 - 1) x 3 = 3 x 2^32 - 3, so the 64-bit result is **0x00000002FFFFFFFD**. HI = **0x00000002**; LO = **0xFFFFFFFD**. It does not fit unsigned 32 bits because HI is nonzero.

```
multu $s0, $s1      # the stated inputs are in s0 and s1
mfhi $t0
mflo $t1
```

The ordinary registers now preserve the halves even if a later multiply/divide changes HI/LO.

Q56. Signed multiplication fit

Related lesson: 3.7.

-3 x 7 = **-21**. Its 64-bit two's-complement pattern is **0xFFFFFFFFFFFFFFEB**, so HI = 0xFFFFFFFF and LO = 0xFFFFFFEB. Signed -21 fits 32 bits. LO's sign bit is 1, and HI is its correct all-one sign extension. Testing HI != 0 would falsely report overflow; the signed fit test compares HI with the sign extension of LO.

Q57. Signed quotient and remainder

Related lessons: 3.8 and 3.10.

Magnitude division 20/6 gives quotient 3, remainder 2. Signed truncation toward zero makes quotient **-3** and remainder **-2**. Check: 6 x (-3) + (-2) = -20. LO stores -3, pattern 0xFFFFFFFD; HI stores -2, pattern 0xFFFFFFFE. Under divu, 0xFFFFFFEC is unsigned 4,294,967,276. Dividing by 6 gives quotient **715,827,879** and remainder **2**, verified by 6 x 715,827,879 + 2 = 4,294,967,276.

Q58. Binary long division

Related lessons: 3.8-3.9.

Read dividend bits 1, 1, 0, 1 from left to right. Start remainder 0 and divide by decimal 3.

Bit	Brought-down partial R	Decision	R after trial	Quotient prefix
1	1	Too small: 0	1	0
1	3	Subtract 3: 1	0	01
0	0	Too small: 0	0	010
1	1	Too small: 0	1	0100

Quotient **0100 = 4**, remainder **0001 = 1**. Identity: 13 = 3 x 4 + 1. Each brought-down partial remainder is 2 x old remainder + next input bit.

Q59. Special-result registers and division safety

Related lesson: 3.10.

The division overwrites HI/LO, so a later mflo retrieves the division quotient, not the earlier product. Copy a required multiply result to an ordinary register before dividing. A zero-divisor guard can be:

```
beq $s1, $zero, DivideError
div $s0, $s1
mflo $t0
j Done
DivideError:
   # program-specific error handling
Done:
```

The error path intentionally has no invented quotient. Another signed edge case is **-2^31 divided by -1**: the exact +2^31 does not fit signed 32 bits. Core classic div does not supply the automatic signed-overflow exception behavior of add; software needs the required checks.

Q60. Normalize a binary fraction

Related lessons: 3.11-3.12.

5 = 101 in binary. 0.625 = 1/2 + 1/8 = .101, so **5.625 = 101.101 = 1.01101 x 2^2**. Sign S = 0; actual exponent 2; stored exponent E = 2 + 127 = **129**, binary 10000001. Fraction begins **01101** and is padded with zeros to 23 bits. No rounding is needed because the binary expansion terminates within the available precision.

Q61. Decode IEEE-754

Related lesson: 3.13.

Split 0xC1200000 as **1** / **10000010** / **01000000000000000000000**. S = 1, E = 130, actual exponent = 3. Fraction value is 1/4, so significand is 1.25. Decimal value = (-1) x 1.25 x 2^3 = **-10**. E is between 1 and 254, so this is a normal value and the implicit leading 1 is appropriate.

Q62. Encode IEEE-754

Related lesson: 3.14.

Magnitude 2.5 = 10.1 = **1.01 x 2^1**. Sign S = 1. E = 1 + 127 = **128**, binary 10000000. Fraction is **01000000000000000000000**. Join fields:

```
1 | 10000000 | 01000000000000000000000
1100 0000 0010 0000 0000 0000 0000 0000
Hex: 0xC0200000
```

The number is exactly representable because only one fractional binary place beyond the leading 1 is nonzero.

Q63. Reserved floating-point patterns

Related lesson: 3.15.

Word	Classification	Value / reason
0x00800000	Smallest positive normal	E=1, fraction=0: 2^-126
0x00000001	Smallest positive subnormal	E=0, fraction integer=1: 2^-149
0x7F800000	Positive infinity	E=255; fraction=0
0x7FC00000	NaN	E=255; fraction nonzero

The subnormal has an implicit leading **0**, not 1; its scale uses 2^-126 multiplied by fraction 2^-23. This produces 2^-149.

Q64. Ties-to-even rounding

Related lesson: 3.15.

Retain leading 1 plus three fraction bits. For **1.1011000...**, retain 1.101; discarded tail is exactly half an ulp. The retained low bit is 1, so round up to **1.110**, whose low bit is 0. For **1.1001000...**, retain 1.100; again exactly halfway, but the retained low bit is already 0, so keep **1.100**. Ties-to-even does not mean always rounding downward or always rounding upward.

Q65. Floating-point addition steps

Related lesson: 3.16.

1.5 = **1.1 x 2^0**; 0.375 = **1.1 x 2^-2**. Align the smaller exponent: 0.375 = **0.011 x 2^0**. Add significands: 1.100 + 0.011 = **1.111**. It is already normalized, giving 1.111 x 2^0 = **1.875**. The fraction terminates after three bits, so binary32 stores it exactly without a changed rounded value. Its binary32 word is 0x3FF00000.

Q66. Floating-point multiplication steps

Related lesson: 3.17.

-1.5 = -1.1 x 2^0; 2.5 = 1.01 x 2^1. Opposite signs give a negative result. Exponents add to 1; significands multiply: 1.1 x 1.01 = **1.111**, already normalized. Result = **-1.111 x 2^1 = -3.75**. Stored exponents are 127 and 128; candidate result E = 127 + 128 - 127 = **128**. S=1, fraction=111 followed by zeros, so binary32 word is **0xC0700000**. No extra normalization exponent increment is needed here.

Q67. FP comparison and control flow

Related lessons: 3.18-3.19.

One correct no-delay-slot implementation is:

```
c.le.s $f0, $f2
bc1f DividePath
mul.s $f0, $f0, $f2
j Done
DividePath: div.s $f0, $f0, $f2
Done:
```

For (2,3), comparison is true and a becomes **6.0**. For (4,2), it is false and a becomes **2.0**. The comparison uses the original inputs before replacing a. Integer slt interprets ordinary register patterns as integers; it does not decode IEEE sign/exponent/fraction or handle NaN semantics. The FP comparison instead uses FP operands and sets an FP condition bit.

Q68. FP instruction fields

Related lesson: 3.20.

The fields for add.d $f8,$f4,$f6 are opcode **17**, fmt **17**, ft **6**, fs **4**, fd **8**, funct **0**. Their widths are 6/5/5/5/5/6.

```
010001 10001 00110 00100 01000 000000
Word: 0x46262200
```

The classic pairs are destination **$f8/$f9**, first source **$f4/$f5**, second source **$f6/$f7**. The written destination is first in Assembly but is encoded in fd, the field occupying bit positions 10:6.

Solutions | Chapter 4 microarchitecture

Q69. Follow a load through five stages

Related lessons: 4.1-4.4.

IF fetches the instruction at PC and prepares PC+4. ID decodes lw, reads base register **$s0, number 16**, whose value is 200, and sign-extends 12. EX adds 200 + 12 = **212**, the effective byte address. MEM reads word **37** from 212. WB chooses rt **$t0, number 8**, and writes 37. The numbers 16 and 8 name registers; 212 is an address; 37 is the loaded data.

Q70. MUX selection and write enable

Related lessons: 4.2 and 4.5.

Select 0 outputs **88**; select 1 outputs **37**. Neither setting adds the inputs. If RegWrite = 0, the destination register is **unchanged**, even though 37 appears on its candidate write-data wire. Selection determines a candidate value; the write enable determines whether storage updates. This is why unused datapath outputs need not change architectural state.

Q71. Extract fields versus register contents

Related lesson: 4.3.

0x02324020 is **add $t0,$s1,$s2**. Read-register numbers are rs=17 and rt=18; actual read data are **20 and 30**. Selected rd=8 identifies $t0. ALU result is **50**; MemtoReg selects this ALU result, so WB writes 50 to register 8. Register identifiers 17/18 are encoded, but current contents 20/30 are not part of the instruction word.

Q72. Load path and control

Related lessons: 4.4-4.7.

-4 has 16-bit pattern 0xFFFC and sign-extended 32-bit pattern **0xFFFFFFFC**. Address = 0x1000 - 4 = **0x0FFC**. Destination is $t2, number **10**; WB data is **123**.

Signal	Value
RegDst	0, choose rt
ALUSrc	1, choose signed immediate
MemtoReg	1, choose memory data
RegWrite / MemRead	1 / 1
MemWrite / Branch	0 / 0
ALUOp	00; final ALU action 0010, add

The ALU produces an address, not the data value 123.

Q73. Store path and control

Related lessons: 4.4-4.7.

Register read values are base $s0 = 100 and data $t2 = 55. Selected ALU operands are **100 and sign-extended 8**, giving address **108**. The separate store-data path carries **55** to memory. MemWrite=1, MemRead=0, RegWrite=0, ALUSrc=1, ALUOp=00, Branch=0. RegDst and MemtoReg are don't-cares because no register is written. Memory word at 108 becomes 55; both source registers remain unchanged.

Q74. Branch signal versus taken signal

Related lessons: 4.5 and 4.8.

Branch = **1** because the instruction is beq. With unequal inputs, subtraction is nonzero, so Zero = **0** and PCSrc = 1 AND 0 = **0**. Candidate target = 1000 + 4 + 2 x 4 = **1012**, but selected next PC is **1004**. For equal inputs, Zero=1, PCSrc=1, and next PC becomes **1012**. Computing a candidate target does not mean it is taken.

Q75. Two levels of ALU decoding

Related lesson: 4.6.

Instruction	ALUOp	Final ALU operation
lw	00	0010, add
beq	01	0110, subtract
R-format sub	10	0110, from funct 0x22

A load's low 16 bits are an address displacement. ALUOp=00 requests addition directly, so its low six bits are not used as a funct field. The same four-bit subtract control can result from beq's instruction class or sub's decoded R-format function.

Q76. Single-cycle critical path

Related lesson: 4.9.

lw uses 80 + 40 + 70 + 100 + 30 = **320 ps**. sw uses 80 + 40 + 70 + 100 = **290 ps**. R-format uses 80 + 40 + 70 + 30 = **220 ps**. beq uses 80 + 40 + 70 = **190 ps** under the stated simplified path model. The common single-cycle period is the maximum, **320 ps**. Four instructions take **4 x 320 = 1280 ps**, irrespective of their mix within this subset.

Q77. Finite pipeline speedup

Related lessons: 4.10-4.11.

Pipeline period = max stage delay + overhead = 100 + 10 = **110 ps**. Seven instructions in five stages need **7+4 = 11 cycles**, so time = **1210 ps**. Single-cycle time = 7 x 320 = **2240 ps**. Finite speedup = 2240/1210, approximately **1.851**. For a long ideal stream, speedup approaches 320/110, approximately **2.909**. It does not approach 5 because these stages are unbalanced and register overhead is included.

Q78. Which instruction controls cycle 5?

Related lessons: 4.12-4.14.

Stage	Instruction
IF	Fifth add $t4,$s5,$s6
ID	Fourth sw $t3,4($s0)
EX	Third add $t2,$s3,$s4
MEM	Second sub $t1,$s1,$s2
WB	First lw $t0,0($s0)

The ALU currently adds with ALUSrc=0 and final operation 0010. Current data-memory MemRead=0 and MemWrite=0 because the sub does not use data memory. WB selects the first load's **memory result**, MemtoReg=1, and writes t0. The main controller currently decodes the **store in ID**, producing a future MemWrite=1; this is not the current MEM enable.

Q79. Preserve the destination number

Related lesson: 4.13.

Current IF/ID.rt identifies younger $t4, number 12. Using it for WB would send the older load's data to t4 instead of $t0, number 8. Carry the older destination through **ID/EX, EX/MEM, and MEM/WB**, together with its write-back controls. WB must use the MEM/WB destination belonging to the same instruction as its value. This is the corrected parcel-label idea from the slides.

Q80. RAW without forwarding

Related lesson: 4.16.

The add runs IF1, ID2, EX3, MEM4, WB5. Without forwarding, the sub cannot read the new to during its original ID3. It stays in ID through cycles **3, 4, and 5**, then EX6, MEM7, WB8. There are **two extra stalls** and **eight total cycles**, instead of ideal 6. WB writes early enough in cycle 5 for ID to read the updated to that same cycle; otherwise another delay would be required.

Q81. Fill a load-use gap safely

Related lessons: 4.17 and 4.21.

One correct ordering is:

```
lw   $t0, 0($s0)
addi $s2, $s2, 1
add  $t1, $t0, $s1
```

The load's MEM is cycle 4; the add's EX is now cycle 5, so its t0 operand can come from MEM/WB without a bubble. The independent addi changes s2, which is not used by the other shown instructions, and neither changes its source s2. Under the stated valid nonfaulting accesses, moving it preserves the fragment's result. The three instructions drain in **7 cycles**, rather than 8 with an unfilled load-use bubble.

Q82. Forward from two different boundaries

Related lessons: 4.19-4.20.

The producer add computes to in EX cycle 3. The sub needs its first input in EX cycle **4**, so forward from **EX/MEM**, **F1=10**. The or needs to as its second input in EX cycle **5**, so use **MEM/WB**, **F2=01**. The other operand selects remain 00. Both values are ready in time; **no stalls** are needed. Three instructions drain in **3+4 = 7 cycles**.

Q83. Choose the latest forwarded value

Related lesson: 4.20.

EX/MEM wins, giving select **10** for the third add's to source. That boundary carries the second add's update, which includes both the first and second additions. MEM/WB carries the older first add's update and would lose the second contribution. Forward only from an instruction whose **RegWrite is enabled**, whose destination is **not register zero**, and whose value is actually available. An EX/MEM load's address is not its loaded result.

Q84. An immediate load-use pair

Related lesson: 4.21.

Instruction	**1**	**2**	**3**	**4**	**5**	**6**	**7**
lw	IF	ID	EX	MEM	WB	-	-
add	-	IF	ID	ID held	EX	MEM	WB

With forwarding, there is **one bubble** and **seven cycles**. The loaded data is only ready after MEM4, so the consumer's EX must move from cycle 4 to cycle 5. It then uses MEM/WB forwarding. Without forwarding, consumer ID must wait through cycle 5, followed by EX6, MEM7, WB8: **two stalls, eight cycles** under same-cycle WB/ID.

Q85. Repair incorrect stall control

Related lesson: 4.22.

Freezing ID/EX with the load still there prevents the producer from advancing to MEM to obtain its value. Correct action is **PCWrite=0**, **IF/IDWrite=0**, and **zero the controls entering ID/EX** to inject a bubble. The load advances into EX/MEM and then performs its memory access. The consumer remains in IF/ID until it can proceed. Hold the front, not the producer's required progress.

Q86. A field match that is not a source dependency

Related lessons: 4.16 and 4.22.

addi $t0,$s1,1 reads **s1**, not t0. Its rt=t0 field is a destination, so it has **no RAW load-use dependency** on the preceding load's t0. Both writes remain ordered in this pipeline. addi $t2,$t0,1 does read t0 as rs and has an immediate load-use hazard: one stall with forwarding, two without. A decoder that treats every rt as a source would insert a false stall in the first case.

Q87. Branch penalties and the endpoint

Related lessons: 4.23-4.24.

Ideal fill/drain count = 100+4 = 104 cycles. Misprediction cost = 5 x 3 = 15 cycles. Total = **119 cycles**. At 2 ns/cycle, time = **238 ns**. Finite-run cycles per useful instruction = 119/100 = **1.19**. Without the four fill/drain cycles, the long-stream estimate is **1 + (5/100)x3 = 1.15 CPI**. Do not add a penalty for every branch if only the mispredicted ones incur it.

Q88. A one-bit predictor

Related lesson: 4.24.

Occurrence	Prediction before outcome	Actual	Correct?	New state
1	N	T	No	T
2	T	T	Yes	T
3	T	T	Yes	T
4	T	N	No	N

There are **two wrong predictions**, costing **4 extra cycles** at two each. Final state predicts N. The predictor's remembered state changes with outcomes; it does not force the branch to follow its prediction.

Solutions | Chapter 5 memory hierarchy

Q89. Identify locality

Related lessons: 5.1-5.3.

Repeated loop instructions and repeated scalar access show **temporal locality**. A sequential array scan shows **spatial locality**, and can show temporal reuse if rescanned. One-time unrelated scattered nodes show weak locality. Keeping recent blocks and fetching neighbors can therefore serve many accesses from fast small memory. A recently used block can still miss if it was evicted by conflicts or capacity pressure; temporal locality describes an access tendency, not a guarantee of residency.

Q90. Set count and address widths

Related lesson: 5.7.

C=2048 bytes, B=32 bytes, W=2. Lines = 2048/32 = **64**; sets = 64/2 = **32**. Offset = log2(32) = **5 bits**; index = log2(32 sets) = **5 bits**; tag = 32-5-5 = **22 bits**. Associativity divides the number of lines to obtain sets, not the number of bytes per block. Check: 22+5+5=32.

Q91. Map a specific byte address

Related lessons: 5.5-5.8.

0x1234 = **4660**. With 16-byte blocks, block = floor(4660/16) = **291**, offset = **4**. There are 1024/16 = 64 lines, so index = 291 mod 64 = **35**, tag = floor(291/64) = **4**. Starting byte address = 291 x 16 = **4656 = 0x1230**. The block covers 0x1230 through 0x123F.

Q92. Direct-mapped state trace

Related lesson: 5.6.

Block	Index	Tag	Outcome	Changed line
0	0	0	Miss	Line 0 holds block 0
4	0	1	Miss	Line 0 replaces 0 with 4
0	0	0	Miss	Line 0 replaces 4 with 0
1	1	0	Miss	Line 1 holds block 1
4	0	1	Miss	Line 0 replaces 0 with 4
1	1	0	Hit	No data replacement

Final occupied lines are **0 -> block 4, tag 1** and **1 -> block 1, tag 0**. Lines 2 and 3 remain invalid. One hit in six accesses gives **16.667% hit rate**; five accesses miss.

Q93. Valid is part of the hit test

Related lesson: 5.6.

Matching tag with valid=0 is a **miss** because the line is not usable. Valid=1 with the wrong tag is also a **miss** because another block occupies that line. Complete condition is **hit = valid AND (stored_tag == requested_tag)** for the selected line, or any matching valid way in a selected associative set. Random/uninitialized tag bits cannot establish a hit on an invalid line.

Q94. Include metadata storage

Related lesson: 5.7.

There are **64 lines**, 32 sets, 5 offset bits, 5 index bits, and **22 tag bits**. Data uses 2048 x 8 = **16,384 bits**. Per line, tag+valid+dirty = 22+1+1 = **24 bits**; all metadata uses 64 x 24 = **1536 bits**. Total = **17,920 bits = 2240 bytes**, excluding replacement/ECC state as requested. Do not charge tag bits per stored word: one tag identifies an entire line/block.

Q95. Placement choices

Related lesson: 5.15.

Direct mapping: both blocks use index **6** because 6 mod 8 = 14 mod 8 = 6; they cannot coexist there. Two-way with eight total lines has four sets: both map to set **2**, and can occupy its **two different ways**. Fully associative permits any of eight lines, so both can coexist. Lookup needs 1 tag comparison for direct, 2 within the selected 2-way set, and 8 for fully associative parallel search.

Q96. LRU updates on hits

Related lessons: 5.16-5.17.

Request	Outcome	Victim	Oldest -> newest afterward
A	Miss	Empty slot	A
B	Miss	Empty slot	A, B
A	Hit	None	B, A
C	Miss	B	A, C
B	Miss	A	C, B

Final order is **C, B**. LRU evicts B at C because A's hit refreshed A's recency. FIFO would evict A at C because A arrived first; FIFO does not refresh arrival order on a hit. Thus "oldest" must mean oldest last access for LRU, not oldest insertion.

Q97. Dirty write-back behavior

Related lesson: 5.11.

After read fill, cache A=10, valid=1, dirty=0; memory A=10. After store hit, cache A=**20**, valid=1, dirty=**1**; lower memory can still be **10**. Load hit returns **20**. Evicting the dirty block requires writing **20** to lower memory before safely losing the cached update. After replacement, the old A is no longer resident in that line; the new line's state belongs to its replacement block. If merely invalidated after write-back, valid becomes 0.

Q98. Allocation is separate from write propagation

Related lessons: 5.10-5.11.

Under **write-through/no-write-allocate**, the store miss updates lower memory but does not fill this cache. The following load **misses**, fetches the updated block, and fills the cache. Under **write-back/write-allocate**, the store miss first fills the block, modifies its cached value, and sets dirty. The following load **hits** and sees the newer cached value. Propagation policy and miss allocation are separate decisions, even though common designs pair them in particular ways.

Q99. Block-transfer bandwidth

Related lesson: 5.12.

Organization	Fill cycles	Bandwidth
Serial narrow memory	1 + 6x12 + 6x1 = **79**	24/79 = **0.304 B/cycle**
Six-word-wide memory/transfer	1 + 12 + 1 = **14**	24/14 = **1.714 B/cycle**
Six banks, serial returns	1 + 12 + 6 = **19**	24/19 = **1.263 B/cycle**

The wide bus can return the whole six-word block together. Interleaved banks overlap the modeled memory service, but the return bus still sends six words separately. These are bus-cycle counts under the requested slide model.

Q100. Miss contributions to CPI

Related lesson: 5.13.

I-cache extra CPI = 0.01 x 40 = **0.4**. D-cache extra CPI = 0.30 x 0.05 x 40 = **0.6**. Total = 1.5+0.4+0.6 = **2.5 CPI**. For 1000 instructions, cycles = **2500**. At 2 GHz, time = 2500/(2x10^9) = **1.25 microseconds**. The 30% factor is necessary because only those instructions access data memory; instruction fetch is counted once for every instruction.

Q101. AMAT and miss-time conventions

Related lesson: 5.14.

With an **additional** 30 ns penalty, miss service is 2+30=32 ns, so AMAT = 2 + 0.10x30 = **5 ns/access**. With **total** miss service 30 ns, AMAT = 0.90x2 + 0.10x30 = **4.8 ns/access**. Adding a 30 ns total time as if it were extra would double-count the miss's initial lookup. CPI additionally depends on accesses per instruction, base processing, and overlap, so ns/access alone does not determine cycles/instruction.

Q102. Local and global L2 miss rates

Related lesson: 5.18.

Global miss rate = 0.08 x 0.25 = **0.02 = 2**% of original top-level accesses. Local L2 miss rate 25% is conditional on requests that reached L2. AMAT = 1 + 0.08 x (5 + 0.25x100) = 1 + 0.08x30 = **3.4 ns/access**. Equivalently, 1 + 0.08x5 + 0.02x100 = 3.4. Every L1 miss pays L2 lookup; only the global 2% pay the additional memory service.

Q103. More GHz does not halve a fixed memory wait

Related lesson: 5.19.

Processor	Period	Extra penalty	Actual CPI	Time per instruction
2 GHz	0.5 ns	50/0.5 = 100 cycles	1+0.02x100 = 3	3x0.5 = **1.5 ns**
4 GHz	0.25 ns	50/0.25 = 200 cycles	1+0.02x200 = 5	5x0.25 = **1.25 ns**

Speedup = 1.5/1.25 = **1.2**, not 2. The fixed 50 ns miss wait contributes 0.02x50 = 1 ns per instruction to both machines; only the ideal processing portion halves.

Q104. Matrix layout and tile footprint

Related lesson: 5.20.

Incrementing **i** gives adjacent elements in index i+j*n. Three double-precision 16x16 tiles use 3x16x16x8 = **6144 bytes = 6 KiB**. Three full 64x64 matrices use 3x64x64x8 = **98,304 bytes = 96 KiB**. Partial tiles must stop at n to avoid out-of-bounds elements when n is not a tile-size multiple. Contributions from all sk tiles belong to the same final C entry, so resetting its partial sum per inner tile would discard earlier products.

Q105. Virtual-to-physical translation

Related lessons: 5.21-5.22.

4 KiB = 0x1000 bytes. VA 0x3456 has **VPN=3**, offset **0x456 = 1110**. Frame 9 gives PA = 9x0x1000 + 0x456 = **0x9456**. Offset width is **12 bits**. A 32-bit VA has **20 VPN bits**; a 30-bit PA has **18 physical-frame bits**. Translation preserves the same 12 low offset bits.

Q106. Change the page size

Related lesson: 5.21.

16 KiB = **0x4000**. Since 0x3456 < 0x4000, VPN = **0** and offset = **0x3456 = 13,398**. Frame 2 starts at 2x0x4000 = 0x8000, so PA = **0xB456**. Offset width is 14 bits. The larger page groups the address with a different VPN and offset; the 4 KiB answer used a different mapping and cannot simply be copied.

Q107. A flat page-table size

Related lesson: 5.22.

8 KiB = 2^13 bytes, giving **13 offset bits**. VPN width = 32-13 = **19 bits**, so a complete flat table has **2^19 = 524,288 entries**. With eight-byte PTEs, size = 2^19x8 = **4,194,304 bytes = 4 MiB**. For 36-bit physical addresses, frame-number width = 36-13 = **23 bits**. The table-size calculation includes every possible VPN, regardless of how many pages a particular program currently uses.

Q108. Three different misses

Related lesson: 5.23.

A TLB miss with resident page requires obtaining the page-table mapping and refilling the TLB; it does not need a disk-backed page fetch. A valid nonresident disk-backed page fault requires OS frame allocation/replacement, page input, mapping updates, and restart. A data-cache miss means translation succeeded but the physical data block is absent from that cache, so fetch it from the next level. **A TLB hit and data-cache miss can occur together**: the translation exists, the nearby data copy does not.

Q109. A small TLB LRU trace

Related lessons: 5.23 and 5.25.

Initial LRU order is [3,1]. All requested pages are resident.

VA	VPN / offset	TLB event	PA	LRU order afterward
0x1234	1 / 0x234	Hit	0x5234	3,1
0x3456	3 / 0x456	Hit	0x9456	1,3
0x2345	2 / 0x345	Miss; evict 1	0x7345	3,2
0x1456	1 / 0x456	Miss; evict 3	0x5456	2,1

Final entries are **VPN 2 -> frame 7**, oldest, and **VPN 1 -> frame 5**, newest. There are two hits, two misses, and **zero page faults**. Evicting VPN 1 from TLB does not remove its page-table mapping or physical page.

Q110. A dirty page victim

Related lesson: 5.24.

First preserve the dirty VPN 3 data by writing frame 5's old contents to its backing location. Remove/update the old resident mapping and stale translation entries consistently. Read VPN 7's page into frame 5, install its permitted PTE and required translation state, and mark the new clean input page appropriately. Then restart the instruction that faulted on VPN 7. For a clean victim with a valid backing copy, its data need not be written back before reuse. Merely overwriting a dirty victim would lose program changes.

Q111. Classify the three Cs

Related lesson: 5.26.

In the two-line fully associative case **A, B, C, A**, the first A/B/C requests are compulsory misses. After C, LRU has evicted A; the final A is a **capacity miss**, since three active distinct blocks compete for only two lines without placement restrictions.

In four-line direct mapping **0, 4, 0**, first 0 and first 4 are compulsory misses. Both use line 0. The final 0 is a **conflict miss**: a fully associative four-line cache could have retained both blocks. The request for first-ever block 4 remains compulsory even though it causes a conflict eviction.

Q112. Combine translation and a data-cache trace

Related lessons: 5.6-5.8 and 5.21-5.25.

All pages are resident. Translation replaces VPN while preserving the page offset. Then split the **physical** address for the data cache, whose four lines and 16-byte blocks give two index bits and four offset bits.

VA	VPN	TLB	PA	Physical block	Cache index / tag / offset	Data cache
0x0034	0	Miss, fill 0	0x2034	515	3 / 128 / 4	Miss
0x0038	0	Hit	0x2038	515	3 / 128 / 8	Hit
0x1034	1	Miss, fill 1	0x3034	771	3 / 192 / 4	Miss, replace 515
0x0034	0	Hit	0x2034	515	3 / 128 / 4	Miss, replace 771

Final cache line 3 is valid, tag **128**, holding block **515**, bytes **0x2030..0x203F**. Lines 0,1,2 remain invalid. TLB final oldest-to-newest order is **1,0**, with mappings 1->3 and 0->2. TLB hit rate = **2/4 = 50%**; data-cache hit rate = **1/4 = 25%**. **Zero page faults** occur: TLB misses retrieve resident mappings. The last access is a translation hit but a data-cache conflict miss.